COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Jul 24, 2026	**DATE SUBMITTED:**	Aug 6, 2026
NAME:	Bal, Kevin Ranz J.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HANDS ON ACTIVITY 3.1 LINKED LISTS

1. PROCEDURE OUTPUT

ILO A: Construct C++ code for a singly and doubly linked list in C++

Screenshot

Singly List

```
0x6ad070
0x6ab060
0x6ab0a0
0
Testing of traversal
K->R->J->B
Testing of sllinsertHead
Q->K->R->J->B
Testing of sllGeneralInsert
Q->K->R->X->J->B
Testing of sllinsertEnd
Q->K->R->X->J->B->U
Testing of deleting the node
Q->K->R->X->J->B
Deleting all nodes
The list is empty
PS C:\Users\solvendar\OneDrive\Dokumen\CPE21S4_BAL>
```

Double List

```
C P E
REVERSE
E P C
Testing the dllInsertHead
FORWARD
X C P E
REVERSE
E P C X
Testing the dllInsertEnd
FORWARD
X C P E 0
REVERSE
0 E P C X
Testing the dllGenIns
FORWARD
X C P E E 0
REVERSE
0 E E P C X
Testing the dllDelNode
FORWARD
X P E E 0
REVERSE
0 E E P X
Deleting all nodes
The list is empty:
PS C:\Users\solvendar\OneDrive\Dokumen\CPE21S4_BAL>
```

Discussion	The Singly contains one only pointer and can only travel in one path and go to the next node and can't go back to its original position, while the Doubly Lists contains two pointers and can go backward and forward navigation.

Table 1-1. Output of Initial/Simple Implementation

Operation	**Screenshot**
Traversal	

```cpp
template <typename T>
void ListTraversal(SingleList<T> *head)
{

    if (head == nullptr)
    {
        std::cout << "The list is empty";
        return;
    }

    while (head != nullptr)
    {
        // PRINT data OF n
        std::cout << head->data;

        if (head->next != nullptr)
        {
            std::cout << "->";
        }

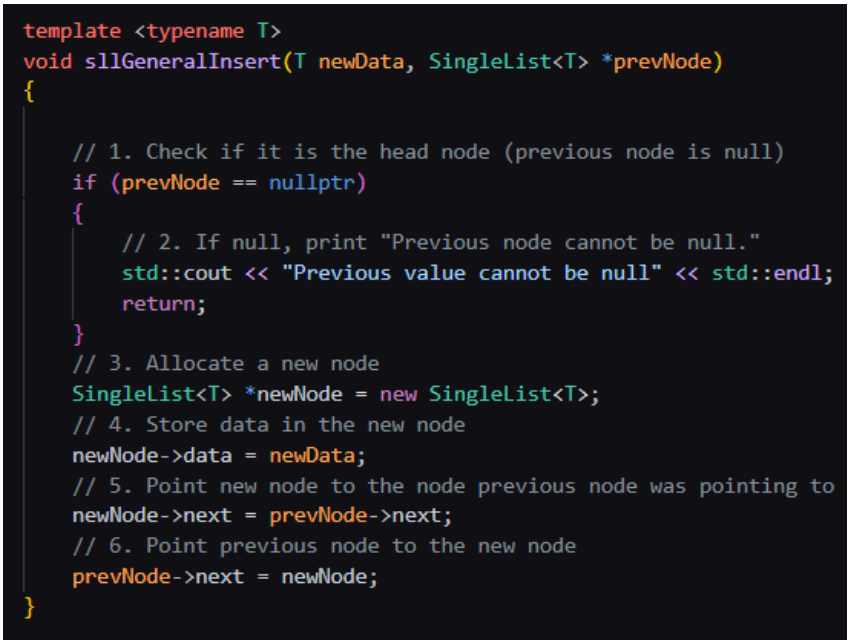
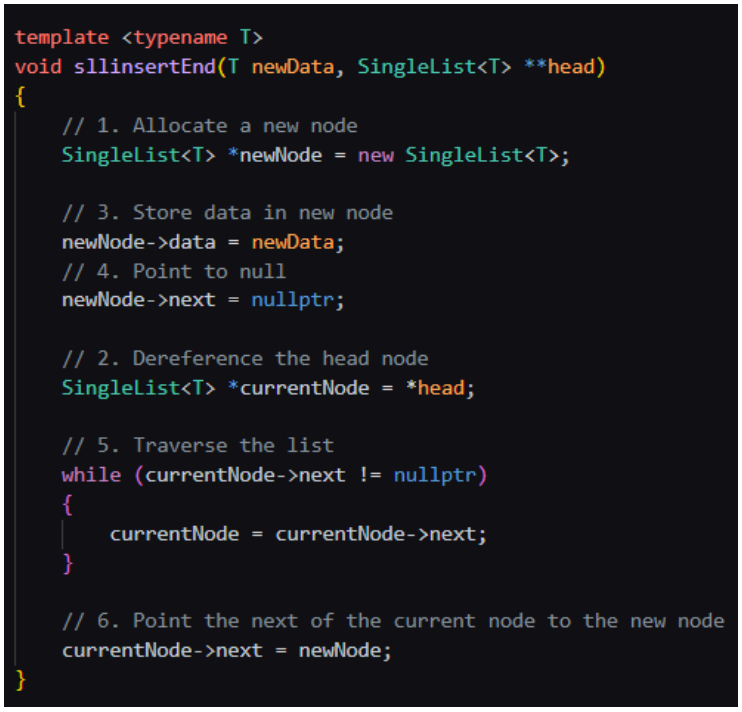
        // GO TO NEXT NODE n -> next
        head = head->next;
    }

    std::cout << std::endl;
}
```

Insertion at Head	

```cpp
template <typename T>
void sllinsertHead(T newData, SingleList<T> **head)
{

    // 1. Allocate new memory for the new code
    SingleList<T> *newNode = new SingleList<T>;
    // 2. Put our data into the new node
    newNode->data = newData;
    // 3. Set next to the new node to point to the previous head
    newNode->next = *head;
    // 4. Reset head to the new node
    *head = newNode;
}
```

Insertion at any part of the list	
Insertion at the end	

```cpp
template <typename T>
void sllGeneralInsert(T newData, SingleList<T> *prevNode)
{

    // 1. Check if it is the head node (previous node is null)
    if (prevNode == nullptr)
    {
        // 2. If null, print "Previous node cannot be null."
        std::cout << "Previous value cannot be null" << std::endl;
        return;
    }
    // 3. Allocate a new node
    SingleList<T> *newNode = new SingleList<T>;
    // 4. Store data in the new node
    newNode->data = newData;
    // 5. Point new node to the node previous node was pointing to
    newNode->next = prevNode->next;
    // 6. Point previous node to the new node
    prevNode->next = newNode;
}
```

```cpp
template <typename T>
void sllinsertEnd(T newData, SingleList<T> **head)
{
    // 1. Allocate a new node
    SingleList<T> *newNode = new SingleList<T>;

    // 3. Store data in new node
    newNode->data = newData;
    // 4. Point to null
    newNode->next = nullptr;

    // 2. Dereference the head node
    SingleList<T> *currentNode = *head;

    // 5. Traverse the list
    while (currentNode->next != nullptr)
    {
        currentNode = currentNode->next;
    }

    // 6. Point the next of the current node to the new node
    currentNode->next = newNode;
}
```

Deletion of a node	

```
template <typename T>
void sllDeleteNode(T findData, SingleList<T> **head)
{

    // if there is nothing to delete
    if (*head == nullptr)
        return;

    SingleList<T> *currNode = *head;
    SingleList<T> *prevNode = nullptr;

    // searching the node: 1. the node does not exist + we found data
    while (currNode != nullptr && currNode->data != findData)
    {
        prevNode = currNode;
        currNode = currNode->next;
    }

    // if data not found currNode == nullPtr in the while loop
    if (currNode == nullptr)
    {
        std::cout << "the data was not found\n"
                  << std::endl;
        return;
    }

    // if data was found
    if (prevNode == nullptr)
    {
        *head = currNode->next;
    }
    else
    {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

Table 1-2. Code for the List Operations

A.	**Source Code**	template <typename T>
		void ListTraversal(SingleList<T> *head){
		if (head == nullptr) {
		std::cout << "The list is empty";
		return;
		}
		while (head != nullptr) {
		// PRINT data OF n
		std::cout << head->data;
		if (head->next != nullptr) {
		std::cout << "->";
		}
		// GO TO NEXT NODE n -> next
		head = head->next;

		} std::cout << std::endl; }
	Console	Testing of traversal K->R->J->B

B.	**Source Code**	template <typename T>
		void sllinsertHead(T newData, SingleList<T> **head) {
		// 1. Allocate new memory for the new code
		SingleList<T> *newNode = new SingleList<T>;
		// 2. Put our data into the new node
		newNode->data = newData;
		// 3. Set next to the new node to point to the previous head
		newNode->next = *head;
		// 4. Reset head to the new node
		*head = newNode;
		}

	Console	Testing of sllinsertHead Q->K->R->J->B

C.	**Source Code**	template <typename T>
		void sllGeneralInsert(T newData, SingleList<T> *prevNode) {
		// 1. Check if it is the head node (previous node is null)
		if (prevNode == nullptr) {
		// 2. If null, print "Previous node cannot be null."
		std::cout << "Previous value cannot be null" << std::endl;
		return;
		}
		// 3. Allocate a new node
		SingleList<T> *newNode = new SingleList<T>;
		// 4. Store data in the new node
		newNode->data = newData;
		// 5. Point new node to the node previous node was pointing to
		newNode->next = prevNode->next;
		// 6. Point previous node to the new node
		prevNode->next = newNode;
		}

	Console	Testing of sllGeneralInsert Q->K->R->X->J->B
D.	**Source Code**	template <typename T> void sllinsertEnd(T newData, SingleList<T> **head) { // 1. Allocate a new node SingleList<T> *newNode = new SingleList<T>; // 3. Store data in new node

		newNode->data = newData;
		// 4. Point to null
		newNode->next = nullptr;
		
		// 2. Dereference the head node
		SingleList<T> *currentNode = *head;
		
		// 5. Traverse the list
		while (currentNode->next != nullptr)
		{
		currentNode = currentNode->next;
		}
		
		// 6. Point the next of the current node to the new node
		currentNode->next = newNode;
		}

	Console	Testing of sllinsertEnd Q->K->R->X->J->B->U

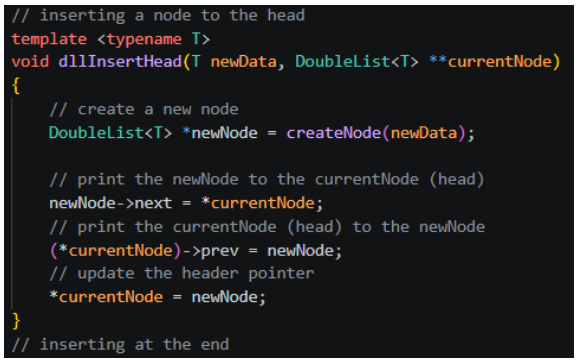
E.	**Source Code**	template <typename T>
		void sllDeleteNode(T findData, SingleList<T> **head) {
		
		// if there is nothing to delete
		if (*head == nullptr)
		Return;
		
		SingleList<T> *currNode = *head;
		SingleList<T> *prevNode = nullptr;
		
		// searching the node: 1. the node does not exist + we found data
		while (currNode != nullptr && currNode->data != findData)
		{
		prevNode = currNode;
		currNode = currNode->next;
		}
		
		// if data not found currNode == nullPtr in the while loop
		if (currNode == nullptr)
		{
		std::cout << "the data was not found\n"
		<< std::endl;
		return;
		}
		// if data was found
		if (prevNode == nullptr) {
		*head = currNode->next;
		} else {
		prevNode->next = currNode->next;

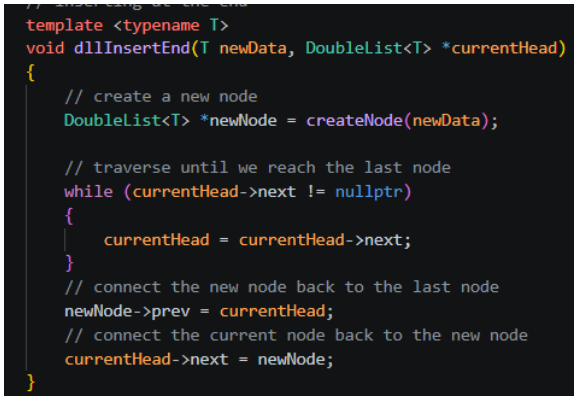
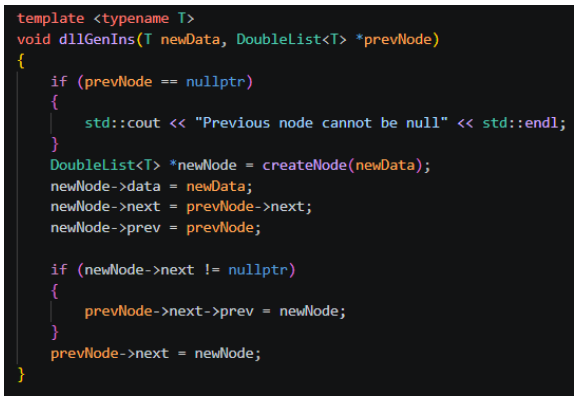
		} delete currNode; }
	Console	Testing of deleting the node Q->K->R->X->J->B

F.	**Source Code**	template <typename T>
		void sllDeleteList(SingleList<T> **head) {
		SingleList<T> *current = *head;
		
		while (current != nullptr) {
		
		SingleList<T> *temp = current;
		current = current->next;
		delete temp;
		}
		
		*head = nullptr;
		}

	Console	Deleting all nodes The list is empty

Table 1-3. Code and Analysis for Singly Linked Lists

Screenshot(s)	Analysis
	The difference between on the singly list is this header doubly uses a “tail” to mark and track both of it first goes through each data from the first to last and uses a prev term variable that has been created on start to go back on the previous node which the doubly can do only.
	This is same on the insert head function of the singly header but uses a previous pointer because it needs to handle the previous nodes because the doubly can go forward and back like to navigate and the double traversal requires to read the list in reverse.

	This function uses a previous to move both forward and backward through the list. Instead of tracking the current node like singly, it used the first node as the starting point to check and to traverse.
	This function inserts a data(char) between the two existing ones and it uses an if statement to check first if the previous node is a null then creates a new node that traverses it using the next and previous pointers.

```
template <typename T>
void dllDelNode(T findData, DoubleList<T> **head)
{
    if (*head == nullptr)
        return;
    DoubleList<T> *currNode = *head;
    DoubleList<T> *prevNode = nullptr;

    while (currNode != nullptr && currNode->data != findData)
    {
        prevNode = currNode;
        currNode = currNode->next;
    }
    if (currNode == nullptr)
        return;
    if (prevNode == nullptr)
    {
        *head = currNode->next;

        if (*head != nullptr)
        {
            (*head)->prev = nullptr;
        }
    }
    else
    {
        prevNode->next = currNode->next;

        if (currNode->next)
        {
            currNode->next->prev = prevNode;
        }
        delete currNode;
    }
}
```

This singly function goes through each node one by one until it finds the node that holds the data you want to remove and once found, it deletes the current node. And if it checks all of the nodes and can't find the data, it will stop and end. Since it's a doubly, every node connects to both, and when deleting it also fixes the two connections properly so the rest of the list stays linked to each of them correctly.

Table 1-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

We'll take the use case of a music player. It should have following functions supported:
Create a playlist using multiple songs.

- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).
-

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

Screenshot of the code

```cpp
#include <iostream>
#include <string>
#include "song.h"

int main() {

    int choice = 0;

    // moved outside the loop so they will not reset to null on every iteration
    SongList<std::string> *head = nullptr;
    SongList<std::string> *currentSong = nullptr;

    while (choice != 6)
    {

        std::cout << "\n=== MUSIC PLAYER MENU ===\n";
        std::cout << "1. Add Song\n";
        std::cout << "2. Remove Song\n";
        std::cout << "3. Next Song\n";
        std::cout << "4. Previous Song\n";
        std::cout << "5. Display All Songs\n";
        std::cout << "6. Exit\n";
        std::cout << "Enter choice: ";
        std::cin >> choice;
        std::cin.ignore(); // clears the trailing newline character from cin

        switch (choice) {

        case 1:
        {

            std::string title;
            std::cout << "Enter song title: ";
            std::getline(std::cin, title); // using the getline to allow spaces in titles

            addSong(&head, title);

            // if nothing is playing yet, it make this new song to the current track
            if (currentSong == nullptr) {
                currentSong = head;
            }
            break;
```

```cpp
        case 2: {

            std::string title;
            std::cout << "Enter song title to remove: ";
            std::getline(std::cin, title);

            // pass &currentSong directly into removeSong so it syncs when a song is deleted
            removeSong(&head, title, &currentSong);
            break;
        }

        case 3:
        {
            nextSong(&currentSong);
            break;
        }
        case 4:
        {
            prevSong(&currentSong);
            break;
        }
        case 5:
        {
            displayPlaylist(head);
            break;
        }
        case 6:
            std::cout << "Exiting Music Player...\n";
            break;

        default:
            std::cout << "Invalid option!\n";
        }
    }

    return 0;
}
```

Output of the code

Adding Song:

```
=== MUSIC PLAYER MENU ===
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 1
Enter song title: Multo

=== MUSIC PLAYER MENU ===
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 1
Enter song title: Judas
```

Choosing display song that added:

```
=== MUSIC PLAYER MENU ===
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 5
 Playlist Layout
- Judas
- Multo
```

Moving to the next and previous song:

```
=== MUSIC PLAYER MENU ===
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 3
Moved to next song: Multo

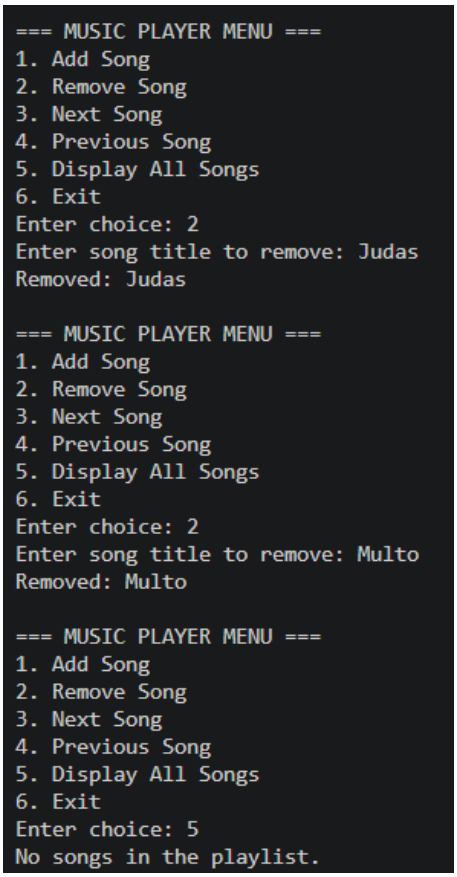
=== MUSIC PLAYER MENU ===
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 4
Moved to previous song: Judas
```

Display all songs:

```
=== MUSIC PLAYER MENU ===
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 5
 Playlist Layout
- Judas
- Multo
```

Removing songs and showing

that there's no display after deleting:



Conclusion

In this activity, fixing multiple bugs in my supplementary part taught me that when it deletes an item from the computer's memory, it has to make sure to update the tracker at the same time. My mistake was moving the playing song tracker too early in one file while actually deleting the song in another file, which left the tracker pointing to an empty, dead memory slot that caused the program to crash. From my mistakes and errors in this code, I learned that whenever I use the delete keyword, any other marker still looking at that spot instantly breaks and points to useless data. Now I know that the safe way to prevent these crashes is to always change the memory and the trackers together inside the same function so they stay matched and never get lost.

Summary of lessons learned	Learned about how linked list work and the difference between the arrays. list can grow and shrink based on what the users what and the arrays have a specific size while list can be complex and inside the list is an independent object/element called node that contains a data or reference or even a pointer to the next node or sequence and doesn't waste a memory because memory is allocated if a new node is created. The downside of it was that it can't jump directly to the index and need to traverse it.
Analysis of the procedure	Learned the difference of both headers that the single one can go only through only one direction and can't go back while the doubly can do both

	because of their two pointer that capable of navigating front and back and also their features and the difference when all of them implemented like next, node, traversal, insertion, deletion of the node and also the list to clean to clean the memory and avoid memory leaking.
Analysis of the supplementary activity	In this program, it is a music player that works like a loop. Each song is linked to the song before it and the song after it, so when it gets to the end of the list, it goes right back to the beginning. In my old code where all of them are totally fine but notice there's something off, when I wanted to delete a song, I tried to move the currently playing song (currentSong) out of the way inside my main driver code before actually deleting it with the removeSong function. This caused a tracking problem for me. If I had two songs left and deleted the one that was playing, the main code moved the player tracker forward too early, while the delete function changed the list behind it. Then, if I tried to delete that last remaining song, the program completely wiped the song from the computer's memory, but the player tracker missed the update and kept looking at that exact, empty memory slot. The next time I pressed next or previous, the program tried to read a song that was already gone, making the whole program crash.I tried to fix it and I learned to pass the player tracker directly into the removeSong function using &currentSong. In this way, the code updates the list and fixes the player tracker at the exact same time inside the same function, making sure the player will not get lost.
Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?	I still can't fully grasp this topic and how to handle them well but I learned how the implementation was done and am still learning from it when it comes to handling the header file and how sensitive it is and what is the cause of segmentation fault even though my code was running. It took me a long process and analysis of where I went wrong when I correctly used the addSong and removeSong functions. From the main driver code that causes an error because I have so many missing inputs because of not passing the song string data to the addSong and removeSong, that's why I try to search on youtube or even at W3School what I am missing to it and it was need an std::getline and allow spaces on the titles in the main driver code. While in the segmentation fault, I didn't notice I created a two song and the first one was playing and the current also pointing to the head when moving to the next which is when i called the removeSong function, it will entirely deletes what is on the head/current which is I changed the remove song where it returns the updated pointer to the list which is why my header file removeSong function have a "`SongList<std::string>* removeSong(SongList<std::string> **head, std::string songTitle, SongList<std::string> *currentSong)`" so when passing the current song directly to the removeSong function, it will sync when the song was deleted.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic

work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Explanation of concepts

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: For the explanation of some coding terms and help in describing where my nodes are travelling to and expanding the explanation about linked list by circular concept part and how it works.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.